---

1. Introduction

In this lab, we implemented and analyzed value-based reinforcement learning methods based on Deep Q-Networks. The experiments were divided into three tasks. Task 1 implemented a vanilla DQN agent for CartPole-v1. Task 2 extended DQN to Atari Pong-v5, where the input observation is a raw image and the Q-network is implemented using a convolutional neural network. Task 3 further enhanced DQN by integrating Double DQN, Prioritized Experience Replay, and multi-step return.

The main findings are as follows. For Task 1, the DQN agent successfully solved CartPole-v1 and achieved a 20-seed average reward of 481.35.

For Task 2, the vanilla DQN learned a stable Pong policy after checkpoint-based continuation and achieved a 20-seed average reward of 16.40. For Task 3, the enhanced DQN was trained from scratch. Although it did not reach the score-19 threshold within 2.5M environment steps, the model improved significantly from near-random play to a best 20-seed average reward of 8.10.

This report first explains the implementation details of DQN, Double DQN, Prioritized Experience Replay, and multi-step return. It then analyzes the training curves and evaluation results for the three tasks. Finally, it discusses sample efficiency, limitations, and possible improvements.

---

2. Implementation

2.1 DQN and Bellman Error

For DQN, the agent stores transitions in the form:

[

(s, a, r, s', done)

]

where (s) is the current state, (a) is the selected action, (r) is the reward, (s') is the next state, and done indicates whether the episode has ended.

The online Q-network estimates the current Q-value:

[
Q_{\text{online}}(s,a;\theta)
]

The target network computes the Bellman target:

[
y = r + \gamma \max_{a'} Q_{\text{target}}(s', a') \cdot (1 - done)
]

The Bellman error is:

[
\delta = y - Q_{\text{online}}(s,a)
]

In the implementation, the online Q-value is obtained by forwarding a batch of states through the Q-network and using gather() to select the Q-value corresponding to the executed action. The Bellman target is computed inside torch.no_grad() so that gradients do not flow into the target network. For Task 1 and Task 2, the DQN loss is computed between the predicted Q-value and the Bellman target.

For CartPole, the Q-network is a multilayer perceptron because the state is a 4-dimensional vector. For Pong, the Q-network is a convolutional neural network because the input is a stack of preprocessed image frames.

2.2 Atari Preprocessing

For Pong-v5, the raw RGB image observation is preprocessed before being passed into the Q-network. The preprocessing procedure includes:

1. converting the RGB frame to grayscale,
2. resizing the image to (84 \times 84),
3. stacking the most recent 4 frames.

The stacked state has shape:

[
4 \times 84 \times 84
]

Frame stacking is necessary because a single image does not contain enough motion information. By stacking consecutive frames, the agent can infer the ball direction and velocity.

2.3 Double DQN

Vanilla DQN uses the target network both to select and evaluate the next action:

[
\max_{a'} Q_{\text{target}}(s', a')
]

This can cause overestimation bias. Double DQN reduces this issue by separating action selection and action evaluation.

The online network selects the greedy next action:

[
a^* = \arg\max_{a'} Q_{\text{online}}(s', a')
]

Then, the target network evaluates this action:

[
Q_{\text{target}}(s', a^*)
]

Therefore, the Double DQN target becomes:

[
y = r + \gamma Q_{\text{target}}(s', \arg\max_{a'} Q_{\text{online}}(s', a')) \cdot (1 - done)
]

In the implementation, q_net(next_states).argmax() is used to select the next action, and target_net(next_states).gather() is used to evaluate the selected action.

2.4 Prioritized Experience Replay

In Task 3, the uniform replay buffer is replaced by a Prioritized Replay Buffer. Instead of sampling all transitions with equal probability, transitions with larger TD errors are sampled more frequently.

For each transition (i), the priority is defined as:

[
p_i = |\delta_i| + \epsilon
]

where (\delta_i) is the TD error and (\epsilon) is a small constant used to avoid zero priority.

The sampling probability is:

[
P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}
]

where (\alpha) controls how strongly prioritization is applied.

Since prioritized sampling changes the data distribution, importance-sampling weights are used:

[
w_i = (N \cdot P(i))^{-\beta}
]

where (N) is the current replay buffer size and (\beta) controls the amount of bias correction. The weights are normalized by dividing by the maximum weight in the sampled batch.

In the implementation, new transitions are inserted with the current maximum priority so that newly collected transitions have a chance to be sampled. After every gradient update, the priorities of sampled transitions are updated using their latest absolute TD errors.

In our implementation, (\alpha = 0.6) and (\beta = 0.4). Due to limited computation time, (\beta) was fixed instead of annealed toward 1.0.

2.5 Multi-Step Return

Vanilla DQN uses a 1-step return:

[
y = r_t + \gamma Q(s_{t+1}, a')
]

In Task 3, we used a 3-step return to propagate delayed rewards more quickly. The n-step return is:

[
R_t^{(n)} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots + \gamma^{n-1}r_{t+n-1}
]

For (n = 3), the target becomes:

[
y = R_t^{(3)} + \gamma^3 Q_{\text{target}}(s_{t+3}, a^*) \cdot (1 - done)
]

In the implementation, a deque is used as the n-step buffer. Each environment transition is first stored in this temporary buffer. Once the buffer reaches length 3, the accumulated discounted reward is computed and the resulting n-step transition is inserted into the replay buffer. At the end of an episode, the

remaining transitions are flushed into the replay buffer so that final experiences are not discarded.

2.6 Weights & Biases Tracking

Weights & Biases was used to track the training and evaluation progress. The logged metrics include:

- environment step count,
- update count,
- episode reward,
- episode length,
- epsilon value,
- evaluation reward,
- evaluation mean reward,
- evaluation standard deviation.

For Task 3, evaluation mean reward and evaluation standard deviation were logged over multiple evaluation episodes. The main x-axis used in the analysis is the environment step count, because the grading criterion is based on sample efficiency, i.e., how many environment steps are required to reach a target Pong score.

Training loss, Q-value mean, and Q-value standard deviation were also printed periodically in the terminal to monitor numerical stability.

---

3. Experimental Results

3.1 Task 1: CartPole-v1



For Task 1, the vanilla DQN successfully solved CartPole-v1. The training curve shows that the total reward and evaluation reward increased over training and eventually reached the maximum environment reward of 500.

The final 20-seed evaluation result is shown below.

- 各種子（Seed）的獎勵分數：
  - Seed 0: 371.0
  - Seed 1: 488.0
  - Seed 2: 500.0
  - Seed 3: 500.0
  - Seed 4: 500.0
  - Seed 5: 500.0
  - Seed 6: 456.0
  - Seed 7: 500.0
  - Seed 8: 494.0
  - Seed 9: 500.0
  - Seed 10: 500.0
  - Seed 11: 500.0
  - Seed 12: 420.0
  - Seed 13: 500.0
  - Seed 14: 500.0
  - Seed 15: 500.0
  - Seed 16: 500.0
  - Seed 17: 500.0
  - Seed 18: 398.0
  - Seed 19: 500.0
- 平均獎勵：481.35
- 標準差：37.82
- 最低獎勵：371.00

To summarize:

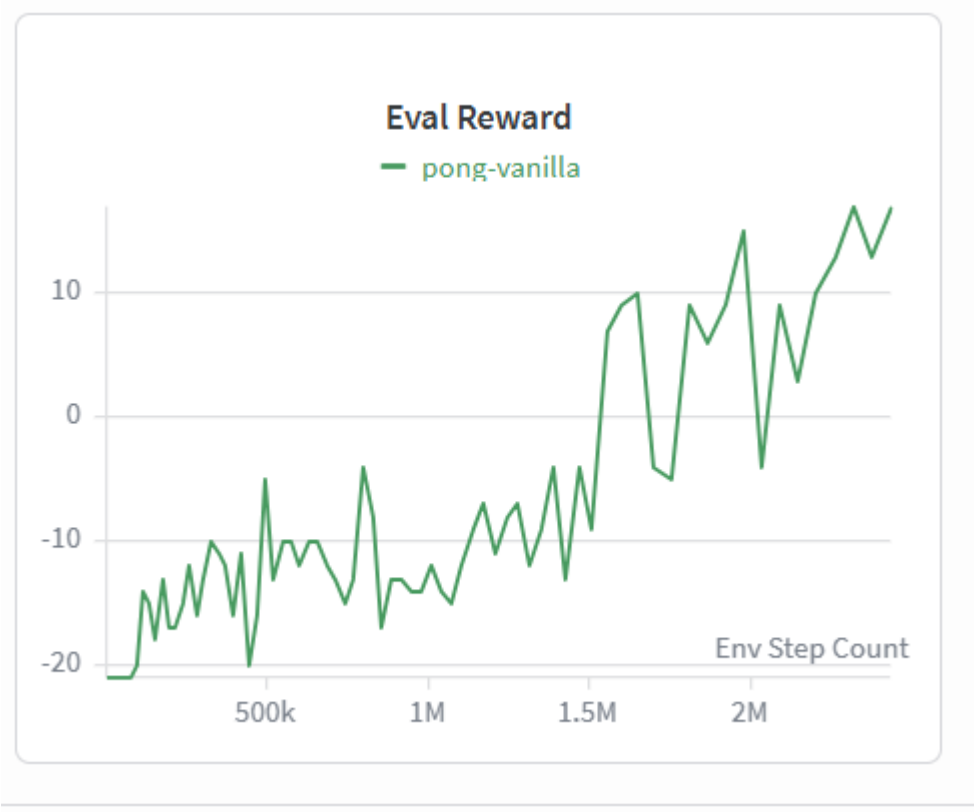
Metric	Value
Average reward	481.35

Metric	Value
Standard deviation	37.82
Minimum reward	371.00
Maximum reward	500.00

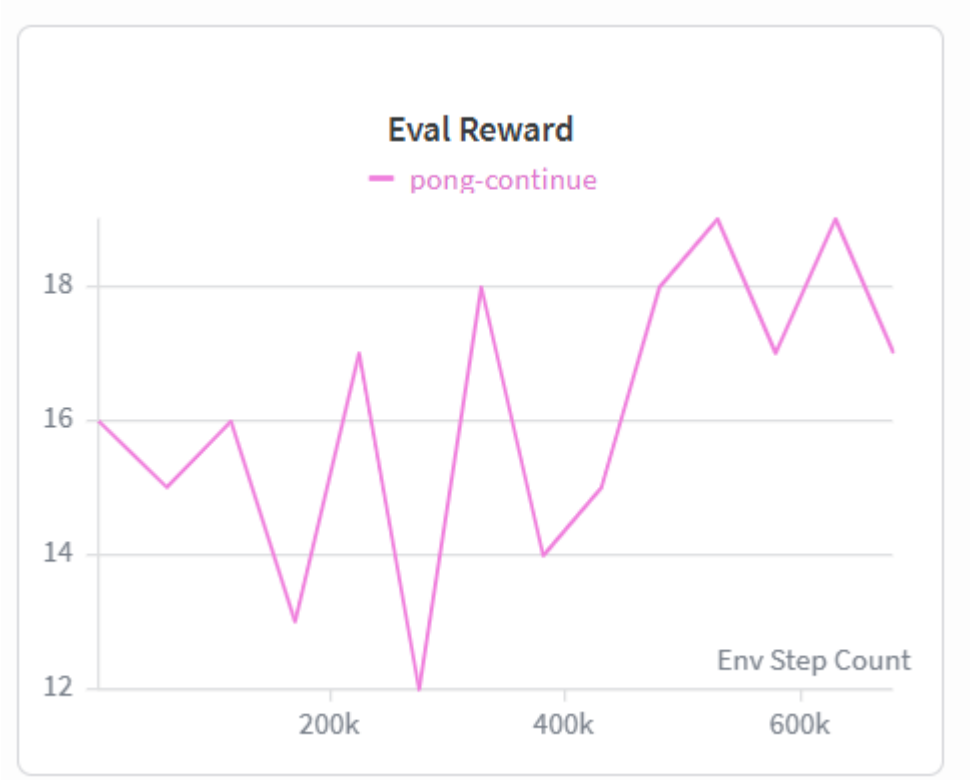
The average reward is above 480, which satisfies the full-score threshold for Task 1. Although a few seeds resulted in lower scores, most evaluation episodes reached the maximum reward of 500.

3.2 Task 2: Vanilla DQN on Pong-v5

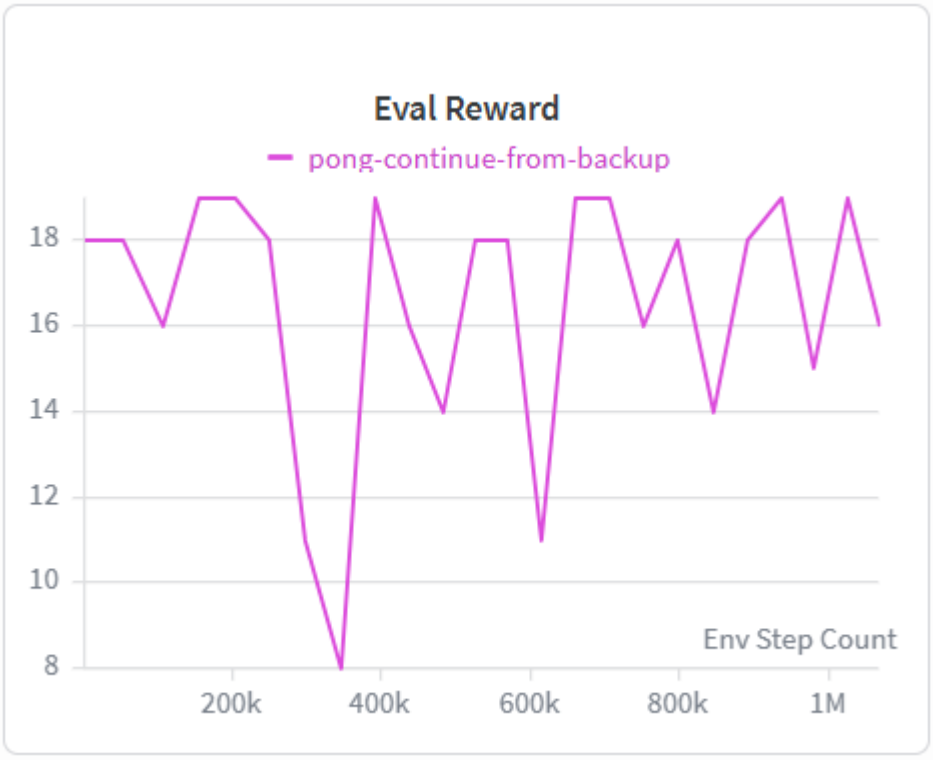
Task 2 initial Pong training curve:



Task 2 first continuation curve:



Task 2 backup continuation curve:



For Task 2, the vanilla DQN was trained on Pong-v5 using raw image observations. The input frames were converted to grayscale, resized to (84 \times 84), and stacked over 4 frames. A convolutional Q-network was used to predict Q-values for all Pong actions.

Due to Colab usage limits, the Task 2 training was conducted across multiple checkpoint-based sessions. The first session was trained from scratch. After the Colab quota was exhausted, training was resumed from saved checkpoints on another account. Only the model weights were restored during continuation; the replay buffer, optimizer state, update counter, and environment state were reinitialized.

The first training session improved the evaluation reward from approximately -20 to positive values. The continuation sessions used lower epsilon values and smaller learning rates, which acted as fine-tuning stages. As a result, the evaluation reward became more stable and frequently reached the 16–19 range.

The final selected Task 2 model was evaluated over 20 seeds.

```
A.L.E: Arcade Learning Environment (version 0.11.2+ecc1138)
[Powered by Stella]
Seed 0: reward = 17.0
Seed 1: reward = 15.0
Seed 2: reward = 19.0
Seed 3: reward = 13.0
Seed 4: reward = 16.0
Seed 5: reward = 18.0
Seed 6: reward = 18.0
Seed 7: reward = 11.0
Seed 8: reward = 16.0
Seed 9: reward = 15.0
Seed 10: reward = 17.0
Seed 11: reward = 18.0
Seed 12: reward = 15.0
Seed 13: reward = 18.0
Seed 14: reward = 14.0
Seed 15: reward = 18.0
Seed 16: reward = 19.0
Seed 17: reward = 17.0
Seed 18: reward = 15.0
Seed 19: reward = 19.0
========================================
Average reward: 16.40
Std reward: 2.11
Min reward: 11.00
Max reward: 19.00
```

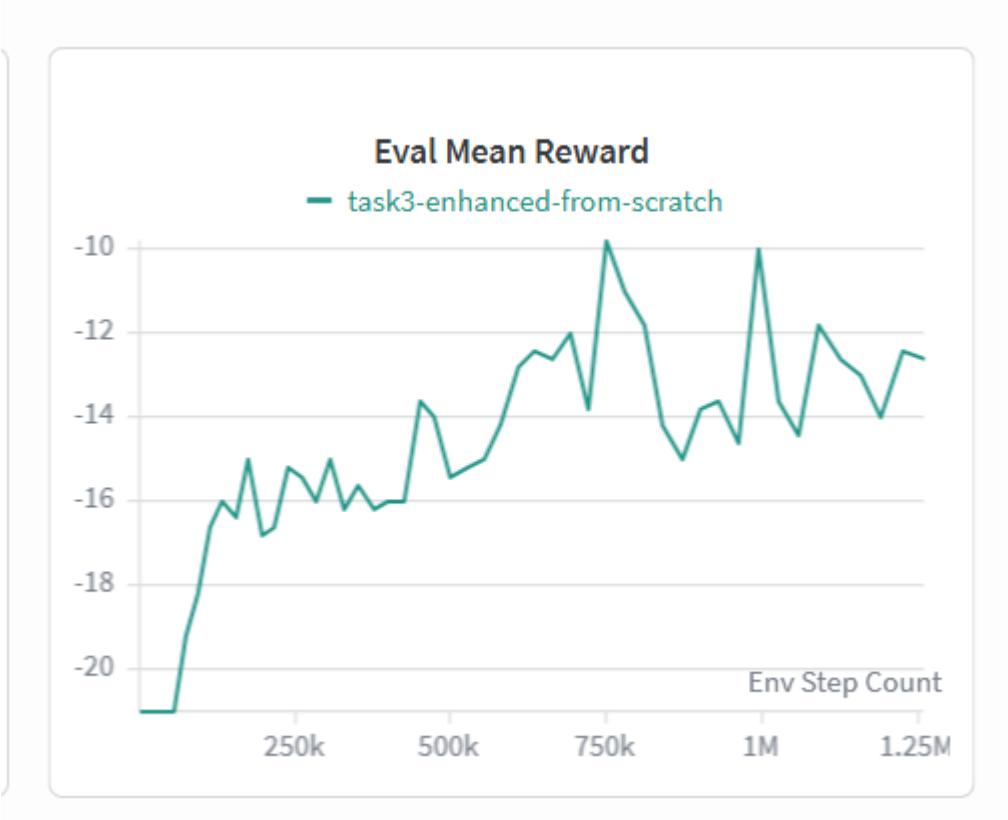
To summarize:

Metric	Value
Average reward	16.40
Standard deviation	2.11
Minimum reward	11.00
Maximum reward	19.00

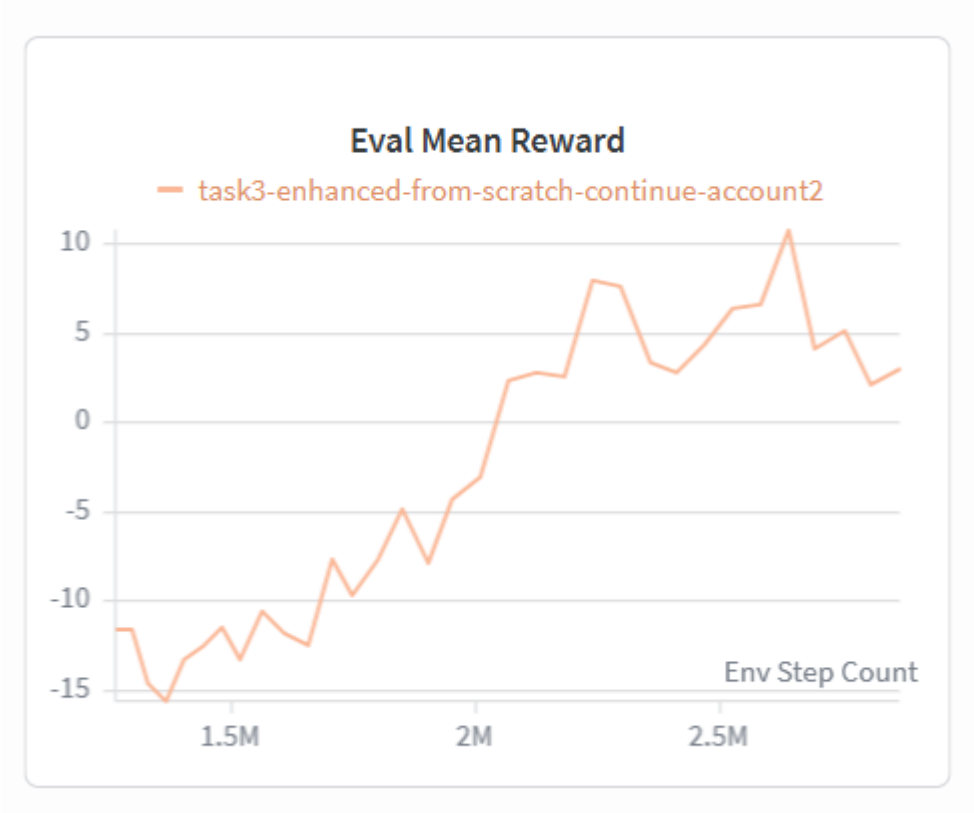
The model achieved a strong Pong policy, with all seeds scoring at least 11 and some seeds reaching 19. However, it did not consistently exceed the score-19 threshold required for full performance score.

3.3 Task 3: Enhanced DQN

Task 3 enhanced DQN first session Eval Mean Reward:



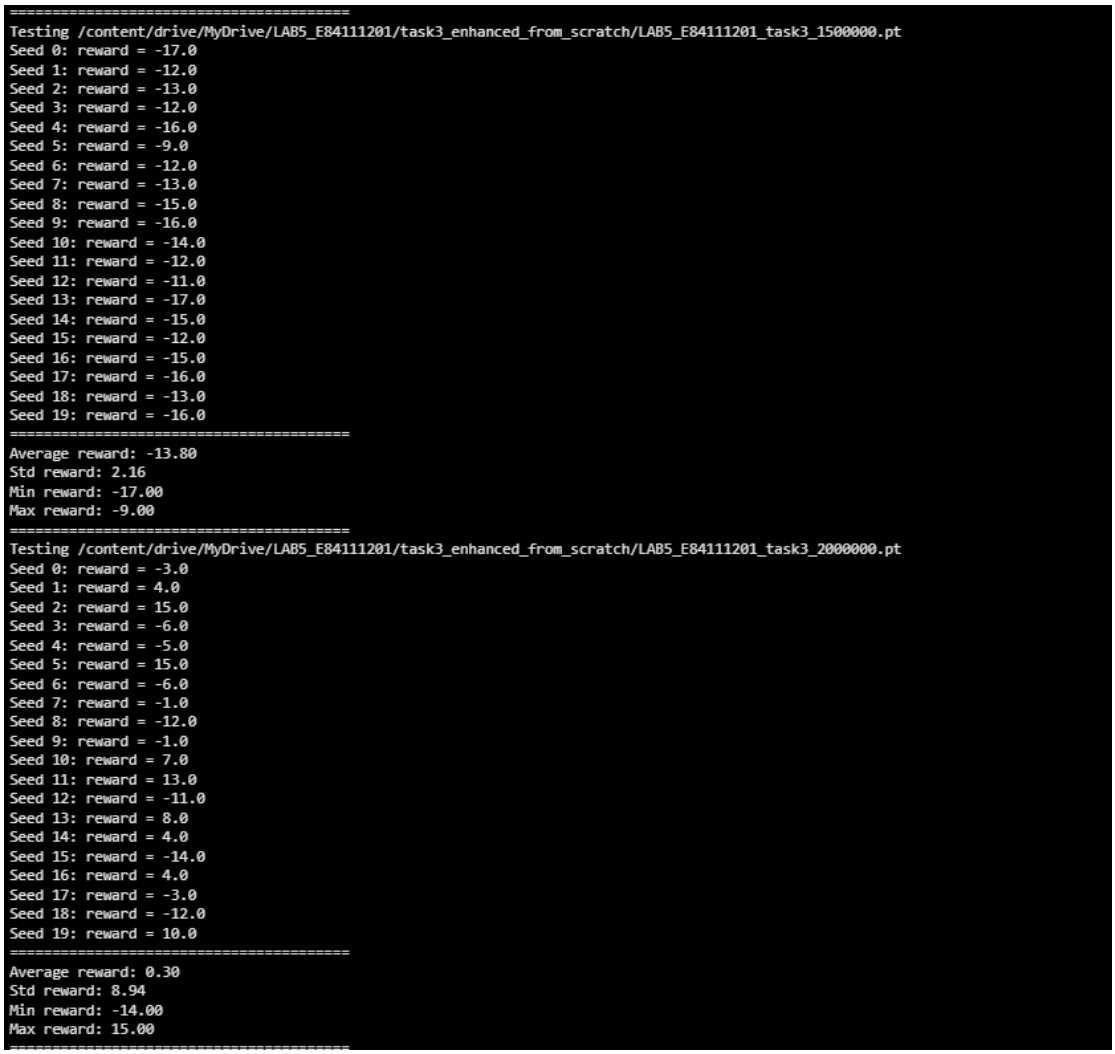
Task 3 enhanced DQN continuation Eval Mean Reward:



For Task 3, the DQN was enhanced using Double DQN, Prioritized Experience Replay, and 3-step return. The model was trained from scratch and later resumed from latest_model.pt due to Colab usage limits. In the second session, the global environment-step counter was continued from 1.26M steps using start-env-count.

The Task 3 training curves show two stages. In the first session, the evaluation mean reward improved from approximately -21 to around -12 by 1.26M environment steps. In the second session, the reward improved more quickly, especially after 2M steps. The evaluation mean reward eventually reached positive values, showing that the agent learned useful Pong behavior.

```
========================================
Testing /content/drive/MyDrive/LAB5_E84111201/task3_enhanced_from_scratch/LAB5_E84111201_task3_600000.pt
Seed 0: reward = -11.0
Seed 1: reward = -13.0
Seed 2: reward = -10.0
Seed 3: reward = -11.0
Seed 4: reward = -11.0
Seed 5: reward = -13.0
Seed 6: reward = -15.0
Seed 7: reward = -7.0
Seed 8: reward = -6.0
Seed 9: reward = -11.0
Seed 10: reward = -15.0
Seed 11: reward = -11.0
Seed 12: reward = -17.0
Seed 13: reward = -15.0
Seed 14: reward = -14.0
Seed 15: reward = -16.0
Seed 16: reward = -16.0
Seed 17: reward = -11.0
Seed 18: reward = -14.0
Seed 19: reward = -15.0
========================================
Average reward: -12.60
Std reward: 2.89
Min reward: -17.00
Max reward: -6.00
========================================
Testing /content/drive/MyDrive/LAB5_E84111201/task3_enhanced_from_scratch/LAB5_E84111201_task3_1000000.pt
Seed 0: reward = -16.0
Seed 1: reward = -15.0
Seed 2: reward = -14.0
Seed 3: reward = -15.0
Seed 4: reward = -6.0
Seed 5: reward = -15.0
Seed 6: reward = -15.0
Seed 7: reward = -15.0
Seed 8: reward = -12.0
Seed 9: reward = -14.0
Seed 10: reward = -15.0
Seed 11: reward = -11.0
Seed 12: reward = -13.0
Seed 13: reward = -12.0
Seed 14: reward = -9.0
Seed 15: reward = -14.0
Seed 16: reward = -17.0
Seed 17: reward = -9.0
Seed 18: reward = -13.0
Seed 19: reward = -17.0
========================================
```



```
========================================
Testing /content/drive/MyDrive/LAB5_E84111201/task3_enhanced_from_scratch/LAB5_E84111201_task3_1500000.pt
Seed 0: reward = -17.0
Seed 1: reward = -12.0
Seed 2: reward = -13.0
Seed 3: reward = -12.0
Seed 4: reward = -16.0
Seed 5: reward = -9.0
Seed 6: reward = -12.0
Seed 7: reward = -13.0
Seed 8: reward = -15.0
Seed 9: reward = -16.0
Seed 10: reward = -14.0
Seed 11: reward = -12.0
Seed 12: reward = -11.0
Seed 13: reward = -17.0
Seed 14: reward = -15.0
Seed 15: reward = -12.0
Seed 16: reward = -15.0
Seed 17: reward = -16.0
Seed 18: reward = -13.0
Seed 19: reward = -16.0
========================================
Average reward: -13.80
Std reward: 2.16
Min reward: -17.00
Max reward: -9.00
========================================
Testing /content/drive/MyDrive/LAB5_E84111201/task3_enhanced_from_scratch/LAB5_E84111201_task3_2000000.pt
Seed 0: reward = -3.0
Seed 1: reward = 4.0
Seed 2: reward = 15.0
Seed 3: reward = -6.0
Seed 4: reward = -5.0
Seed 5: reward = 15.0
Seed 6: reward = -6.0
Seed 7: reward = -1.0
Seed 8: reward = -12.0
Seed 9: reward = -1.0
Seed 10: reward = 7.0
Seed 11: reward = 13.0
Seed 12: reward = -11.0
Seed 13: reward = 8.0
Seed 14: reward = 4.0
Seed 15: reward = -14.0
Seed 16: reward = 4.0
Seed 17: reward = -3.0
Seed 18: reward = -12.0
Seed 19: reward = 10.0
========================================
Average reward: 0.30
Std reward: 8.94
Min reward: -14.00
Max reward: 15.00
========================================
```



```
========================================
Testing /content/drive/MyDrive/LAB5_E84111201/task3_enhanced_from_scratch/LAB5_E84111201_task3_2500000.pt
Seed 0: reward = 5.0
Seed 1: reward = -5.0
Seed 2: reward = 9.0
Seed 3: reward = -3.0
Seed 4: reward = 7.0
Seed 5: reward = 7.0
Seed 6: reward = 9.0
Seed 7: reward = 13.0
Seed 8: reward = 13.0
Seed 9: reward = -5.0
Seed 10: reward = 11.0
Seed 11: reward = 3.0
Seed 12: reward = 12.0
Seed 13: reward = -12.0
Seed 14: reward = 15.0
Seed 15: reward = 7.0
Seed 16: reward = 5.0
Seed 17: reward = 14.0
Seed 18: reward = 7.0
Seed 19: reward = 5.0
========================================
Average reward: 5.85
Std reward: 7.02
Min reward: -12.00
Max reward: 15.00
========================================
```

The following table summarizes the 20-seed evaluation results of the required

Task 3 snapshots.

Checkpoint	Average	Std	Min	Max
600k	-12.60	2.89	-17	-6
1M	-13.35	2.76	-17	-6
1.5M	-13.80	2.16	-17	-9
2M	0.30	8.94	-14	15
2.5M	5.85	7.02	-12	15
Best	8.10	7.19	-9	18

The results show that the enhanced DQN did not learn a strong policy in the early phase. The 600k, 1M, and 1.5M checkpoints still had negative average rewards. However, a clear improvement appeared after 2M steps, where the average reward increased to 0.30. At 2.5M steps, the average reward further improved to 5.85. The best checkpoint achieved an average reward of 8.10 and a maximum reward of 18.

Although the enhanced DQN did not reach the score-19 threshold within 2.5M environment steps, the results show a clear learning trend from near-random play to a positive-reward Pong policy.

---

4. Analysis and Discussion

4.1 Training Curve Analysis

For CartPole, the training process was much faster than Pong because the state space is low-dimensional and the reward signal is dense. The agent receives a reward of 1 for every timestep the pole remains balanced, so the learning signal is frequent. The DQN agent successfully learned a stable policy and reached the maximum reward.

For Pong, the learning process was significantly slower. The input state is high-dimensional image data, and the reward is sparse. Most frames have reward 0,

and the agent only receives +1 or -1 when a point is scored. Therefore, the agent needs many more environment interactions to learn useful behavior.

The vanilla Pong DQN improved gradually during the initial training session and became much stronger after checkpoint-based continuation. The enhanced DQN also showed improvement, especially after 2M environment steps, but it did not reach the same final evaluation score as the vanilla DQN.

4.2 Sample Efficiency Comparison

The final vanilla DQN checkpoint achieved a higher 20-seed average reward than the enhanced DQN checkpoint. However, this comparison is not fully controlled.

The vanilla DQN was trained across multiple continuation sessions and was further fine-tuned with low epsilon values. Its final model therefore benefited from a longer effective training process and more checkpoint selection. In contrast, the enhanced DQN was mainly evaluated under the required 2.5M environment-step setting from scratch.

The enhanced DQN did improve significantly over training. The average reward increased from around -13 in the early snapshots to 5.85 at 2.5M steps, and the best checkpoint reached 8.10. This indicates that the enhanced agent learned useful behavior, but did not reach the score-19 sample-efficiency target.

4.3 Why Enhanced DQN Did Not Outperform Vanilla DQN

Although the enhanced DQN used Double DQN, PER, and 3-step return, it did not outperform the final vanilla DQN checkpoint in our experiments. There are several possible reasons.

First, the vanilla DQN had more effective training and fine-tuning time. The continuation runs used pretrained checkpoints and low epsilon values, which made them more like policy fine-tuning rather than training from scratch.

Second, the enhanced method introduced additional hyperparameters, including PER (\alpha), PER (\beta), n-step return length, epsilon decay

schedule, and learning rate. These parameters were not exhaustively tuned due to limited Colab computation budget.

Third, the first enhanced DQN run used a relatively slow epsilon decay of 0.999999. This kept the agent in a high-exploration regime for a long time and likely delayed early learning. In the continuation run, a faster epsilon decay of 0.999995 was used, and the reward improved more quickly.

Fourth, PER and multi-step return can improve learning, but they can also increase training variance. PER may oversample high-TD-error transitions, which are not always the most useful samples. Multi-step return propagates reward faster, but it also makes the target more variable.

Therefore, the result does not imply that the enhanced techniques are ineffective. Instead, it suggests that these techniques require more careful hyperparameter tuning to consistently outperform vanilla DQN.

4.4 Limitations and Future Improvements

There are several limitations in this experiment.

First, due to Colab usage limits, both Task 2 and Task 3 were trained across multiple sessions. When training was resumed, only model weights were restored. The replay buffer, optimizer state, and update counter were not restored. This means that the continuation was not a full-state recovery.

Second, the comparison between vanilla DQN and enhanced DQN was not perfectly controlled. The vanilla DQN received additional low-epsilon fine-tuning, while the enhanced DQN was mainly evaluated within the required 2.5M-step setting.

Third, the enhanced DQN hyperparameters were not fully tuned. In future work, I would tune the epsilon decay schedule, learning rate, PER parameters, and n-step value more carefully. In particular, (\beta) in PER could be annealed from 0.4 to 1.0 to better correct the bias introduced by prioritized sampling.

Finally, an ablation study would be useful. I would separately evaluate:

- vanilla DQN,
- Double DQN only,
- Double DQN + PER,
- Double DQN + multi-step return,
- Double DQN + PER + multi-step return.

This would make it easier to identify which enhancement contributes most to learning performance.

---

5. Reproducibility Commands

Task 2: Vanilla Pong DQN Initial Training

```
python dqn.py \
    --env-name ALE/Pong-v5 \
    --wandb-run-name pong-vanilla \
    --save-dir ./results_pong_vanilla
```

Task 2: Continuation Training

```
python dqn.py \
    --env-name ALE/Pong-v5 \
    --resume-path /content/drive/MyDrive/Lab5_RL/results_task2/best_model.pt \
    --save-dir /content/drive/MyDrive/Lab5_RL/results_task2_continue \
    --wandb-run-name pong-continue \
    --epsilon-start 0.10 \
    --epsilon-min 0.05 \
    --epsilon-decay 0.999999 \
    --replay-start-size 50000 \
```

```
    --memory-size 100000 \
    --batch-size 32 \
    --lr 0.00005
```

Task 2: Backup Continuation

```
python dqn.py \
    --env-name ALE/Pong-v5 \
    --resume-path
/content/drive/MyDrive/LAB5_E84111201/continue/best_model_reward19_back
up.pt \
    --save-dir /content/drive/MyDrive/LAB5_E84111201/continue_from_backup \
    --wandb-run-name pong-continue-from-backup \
    --epsilon-start 0.05 \
    --epsilon-min 0.03 \
    --epsilon-decay 0.999999 \
    --replay-start-size 20000 \
    --memory-size 100000 \
    --batch-size 32 \
    --lr 0.00005
```

Task 3: Enhanced DQN From Scratch

```
python dqn.py \
    --env-name ALE/Pong-v5 \
    --save-dir
/content/drive/MyDrive/LAB5_E84111201/task3_enhanced_from_scratch \
    --wandb-run-name task3-enhanced-from-scratch \
    --epsilon-start 1.0 \
```

```
    --epsilon-min 0.05 \
    --epsilon-decay 0.999999 \
    --replay-start-size 50000 \
    --memory-size 100000 \
    --batch-size 32 \
    --lr 0.0001 \
    --target-update-frequency 1000 \
    --train-per-step 1
```

Task 3: Enhanced DQN Continuation

```
python dqn.py \
    --env-name ALE/Pong-v5 \
    --resume-path
/content/drive/MyDrive/LAB5_E84111201/task3_enhanced_from_scratch/latest_
model.pt \
    --save-dir
/content/drive/MyDrive/LAB5_E84111201/task3_enhanced_from_scratch \
    --wandb-run-name task3-enhanced-from-scratch-continue-account2 \
    --start-env-count 1260000 \
    --epsilon-start 0.30 \
    --epsilon-min 0.05 \
    --epsilon-decay 0.999995 \
    --replay-start-size 20000 \
    --memory-size 100000 \
    --batch-size 32 \
    --lr 0.0001 \
```

```
    --target-update-frequency 1000 \
    --train-per-step 1
```

Evaluation Command for Pong

```
python fast_test.py \
    --model-path  LAB5_E84111201/continue_from_backup/best_model.pt \
    --episodes 20 \
    --seed-start 0
```

Evaluation Command for CartPole

```
python eval_cartpole.py \
    --model-path <LAB5_E84111201/
LAB5_E84111201_Code/results/best_model.pt> \
    --episodes 20 \
    --seed-start 0
```

---

6. Conclusion

In this lab, we implemented vanilla DQN and enhanced DQN for value-based reinforcement learning. The CartPole task was successfully solved with a 20-seed average reward of 481.35. The vanilla Pong DQN achieved a strong final score of 16.40 after checkpoint-based continuation and fine-tuning. The enhanced DQN showed clear learning progress from near-random behavior to a best average reward of 8.10, but it did not reach the score-19 threshold within 2.5M environment steps.

The results show that DQN can effectively learn both low-dimensional control tasks and image-based Atari tasks. However, the enhanced DQN methods require careful hyperparameter tuning. Future work should include beta annealing for PER, faster and more controlled epsilon schedules, and ablation studies to isolate the contribution of each enhancement.